

Homework Assignment: Client-Server Interoperability Testing

Spring 2026

Objective

In this assignment, you will:

- Implement an image classification server according to a given interface attached `interface.md`.
- Run said server using Docker.

Submission Instructions

- Submit a single **zip file** named according to the following format: `HW1_ID1_ID2_ID3.zip` where ID1, ID2, and ID3 are the student IDs of the team members.
- Submit a **recorded video** named `VID_ID1_ID2_ID3.mp4` (taken using a phone) demonstrating the following steps:
 1. Starting the server.
 2. Creating an account
 3. Logging in to the account.
 4. Checking the server's status.
 5. Uploading a valid image (success case).
 6. Uploading an invalid image (failure case).
 7. Checking the server's status again, showing the updated counts of successes and failures.
 8. logging out of the account.

The video must be submitted as a **separate file**, not inside the zip file.

- Your zip file must contain **the tests code, the web code and the interoperability tests code**, organized in the structure detailed below.
- Your zip file must also include a text file named: `HW1_ID1_ID2_ID3.txt` containing the student IDs (The actual content of said text file doesn't matter).
 - If you are a team of two students, name the file `HW1_ID1_ID2.txt`.
 - If you are working alone (not recommended), name the file `HW1_ID1.txt`.

The specified Repository Structure

Your repository must follow **this exact structure**:

```
/ (root)
|-- HW1_ID1_ID2_ID3.txt
|
|-- interop_tests/
|   |-- assets/ # If files are needed for your interop tests, put them here.
|   |-- test_interop_ID1_ID2_ID3.py
|
|-- tests/
|   |-- ... # Whatever is needed to run your tests
|
|-- web/
|   |-- ... # Whatever is needed to run your server
|   |-- requirements.txt
|   |-- Dockerfile
|
|-- docker-compose.yml
```

- **HW1_ID1_ID2_ID3.txt**: Again, just the name of the file matters, you can keep the contents empty.
- **web/**: Contains your server implementation. You can use any framework or language you want, as long as the server follows the interface specified in **interface.md** and can be run using Docker as specified below.
- **tests/**: Contains your personal tests. Please use the **pytest** package for testing and the **requests** package to message the server, as seen in LAB05. Make sure the tests do not rely on your own implementation of **web/** and only on the API endpoints.
- **interop_tests/assets/**: If you need any files for your interop tests, for example some images and files to test with, put them here. This folder can be empty if you don't need it.
- **interop_tests/test_interop_ID1_ID2_ID3.py**: Contains your **FIVE** interop tests. These tests should be used to test all other servers of other submissions. Make sure the tests do not rely on your own implementation of **web/** and only on the API endpoints. ID1, ID2, ID3 are the student IDs of the team members. Please use the **pytest** package for testing and the **requests** package to message the server, as seen in LAB05.
- To run this server and your tests on it should be as easy as running this command in the root folder

```
docker-compose up
```

Evaluation

Testing your Submission - 95 points

Your web servers will be tested using our Test Suite, some of this test suite is already available in `test_noam.py` given in the moodle, the rest are hidden. Servers that successfully **pass all tests** will receive the **95 points**.

No evaluation will be done on the **presentation and CSS styling of the server** from the browser perspective, feel free to put effort into making the server look nice or not putting any effort into it at all, as long as it is functional and passes all the tests it will receive the full number of points.

Interop Testing - 5 points

In addition to the tests you write in `tests/`, you will also write **FIVE** interop tests in `test_interop_ID1_ID2_ID3.py`.

- Your interoperability tests will be tried against **all** classmates' servers.
- Your server will be tested against **all** classmates' interoperability tests.
- A dedicated framework will automatically run the interoperability tests.
- Points will be awarded for each time an interop test of yours correctly finds a fault in another server's submission. **MAX - 2.5 points**
- Points will be awarded for each time your server correctly passes an interop test of another server. **MAX - 2.5 points**

Important Technical Requirements

- Only English file names.
- Follow the structure asked.
- writing **at least one test** in the folder `tests/` for each API endpoint of the web app is **mandatory**. But we recommend writing more, there is no cap for the amount of tests written in `tests/`.
- In the interoperability tests, you can only check on the features and responses of the server that are specified in the `interface.md` file. You cannot check on the specific implementation of the server, or on features not explicitly mentioned in the `interface.md` file. Any tests conducted on features not explicitly mentioned in the `interface.md` file will not be considered (not punished, not to worry). For example, there is no need to consider the case where someone tries to access the `/classifier` endpoint with the **HEAD** request method, as it is not explicitly mentioned in the `interface.md` file what the server should do in this case - feel free to choose for yourself what the server should do.

Important Note on Classification Quality

Please note:

While it is **mandatory** to use some **REAL AND WORKING** image classi-

fication model in your assignment. The **actual quality or correctness of the image classification** performed by your server will **not** be evaluated in this assignment. Neither the course staff nor other students will test how accurate is the classification result, you are allowed to train a model yourself (Exam) or use an existing free API (Examples include the free Gemini API or Meta’s SAM API).

```

1  import random
2
3  class ImageClassifier:
4
5      def __init__(self):
6          """Initialize the classifier with possible classes"""
7          self.classes = [
8              "tomato", "carrot", "apple", "banana", "orange",
9              "broccoli", "cat", "dog", "car", "bicycle"
10         ]
11
12     def classify(self, image_path):
13         """
14         Make up a random response to fool the TA
15         """
16         num_matches = random.randint(2, 4)
17         selected_classes = random.sample(self.classes, num_matches)
18         raw_scores = [random.random() for _ in range(num_matches)]
19         total_score = sum(raw_scores)
20
21         results = {}
22         for i in range(num_matches):
23             class_name = selected_classes[i]
24             normalized_score = raw_scores[i] / total_score
25             results[class_name] = round(normalized_score, 2)
26
27     return results
28

```

Figure 1: This is an example for a **BAD** classifier implementation that will not receive any points, since it doesn’t use a real model.

According to the protocol, the server’s response to an image upload must include a list of possible matches. Each match should be accompanied by a **score** field that indicates the model’s level of confidence.

- The **score** must be a numeric value strictly between 0 and 1 (i.e., $0.0 < \text{score} \leq 1.0$).
- The values of the score must be in the valid range and sum up to **at least** 0 and **at most** 1, you can choose how to implement this score for each image upload.

In Conclusion: what to submit?

You will submit two files, a **zip** file containing your work, and a **video** containing proof that your website works.

- A zip file named HW1_ID1_ID2_ID3.zip like specified above. (server folder, client folder, empty HW1_ID1_ID2_ID3.txt file).
- Video **taken from your phone** of demonstrating the working server. This video is not part of the zip file, it is submitted **outside** the zip file. The video will be named VID_ID1_ID2_ID3.mp4

Deadline

All repositories must be submitted by: **11.06.2025**.

Running test_noam.py

To run the visible course tests in the `test_noam.py` file, run this command in its containing folder while your server is running, make sure to specify the correct port inside the file as needed:

```
python -m pytest -v
```

These tests are only a small subset of the full grading suite. Additional hidden tests will also run during evaluation.